

GPU PyFR PASS 2 – Robust NASA Hump Execution on Purdue Gilbreth

OpenAI Codex workflow inside `nasa-hump2`

May 12, 2026

1 Objective

GPU PyFR PASS 2 builds directly on PASS 1 and targets one concrete milestone: produce the first valid completed PyFR hump solution on Gilbreth without depending on the PASS 1 sampler stack. The pass is deliberately conservative. The goal is robust completion first, then external postprocessing, and only then carefully staged probe reintroduction.

2 PASS 1 Failure Diagnosis

PASS 1 proved that the environment and Slurm/CUDA path were sound, but it also identified two separate hump-case blockers:

- the earlier curved and structured hump meshes failed during PyFR face-point/interface sorting,
- the only hump mesh that advanced into the solver then failed inside the sampler plugin before exporting valid wall or benchmark CSVs.

The detailed diagnosis is recorded in `documentation_src/gpu_pyfr_pass2/PASS1_FAILURE_DIAGNOSIS.md`.

3 Why Sampler Removal Was the First Fix

Because the linearized triangular hump mesh already advanced past import and solver setup in PASS 1, PASS 2 does not start by adding more probes. Instead, it removes all sampler plugins and keeps only:

- standard solution writing,
- pseudo-stat output,
- integrated wall-force output.

This isolates the remaining failure mechanism and gives PyFR the best chance to complete a hump solution before any point-localization logic is asked to run.

4 Gilbreth Environment Summary

PASS 2 reuses the verified PASS 1 user-space environment on Gilbreth:

- access via `ssh gilbreth`,
- workspace rooted in `scratch` under `repo_git`,

- gcc/11.5.0, openmpi/4.1.6, cuda/12.6.0,
- pyfr 1.15.0,
- Slurm GPU execution on the a100-40gb partition.

5 Mesh and Topology Fixes

PASS 2 preserves the most promising PASS 1 hump mesh family: the piecewise-linear triangular mesh. The changes are procedural rather than geometric:

- keep the lower hump wall and upper contour from the existing reconstruction,
- avoid curved high-order geometry in the first robust hump run,
- audit the generated mesh before submission with `scripts/pyfr/check_pyfr_hump_mesh.py`,
- use order-1 PyFR settings for the first no-sampler execution.

6 Minimal No-Sampler Run

The first PASS 2 submission mode is `minimal`. It uses:

- the medium piecewise-linear triangular hump mesh,
- order 1,
- short final time,
- conservative dual-time settings,
- no sampler plugins.

7 Incremental Sampler Reintroduction

Only after the minimal run completes does PASS 2 climb the sampling ladder:

1. `minimal`: no samplers,
2. `field_only`: a handful of safe interior points,
3. `sparse_wall`: a very small wall-probe set,
4. `full_wall`: broader wall and evaluation points.

Each rung gets its own Slurm log and result directory.

8 PyFR Output and External Postprocessing

PASS 2 moves extraction out of PyFR whenever possible. The postprocessor:

- reads Slurm logs,
- reads pseudo-stat CSV files,
- checks for exported VTU snapshots,
- builds contours externally if the VTU exists,
- reports C_p/C_f only if they become defensible from the actual outputs.

9 Run Status

Case	Status	Final pseudo time	Notes
minimal	pending	–	Valid no-sampler Slurm submission created on Gilbreth, but still pending behind AssocGrpGRES scheduler limits at the time of local PASS 2 packaging.
field_only	not_submitted		Held back intentionally until the no-sampler minimal hump run completes and writes solution output.
sparse_wall	not_submitted		Held back intentionally until the no-sampler minimal hump run completes and external postprocessing confirms a usable solution snapshot.
full_wall	not_submitted		Held back intentionally because PASS 1 showed that broad wall and evaluation sampling is the fragile failure path. PASS 2 only promotes this rung after minimal and sparse tests succeed.

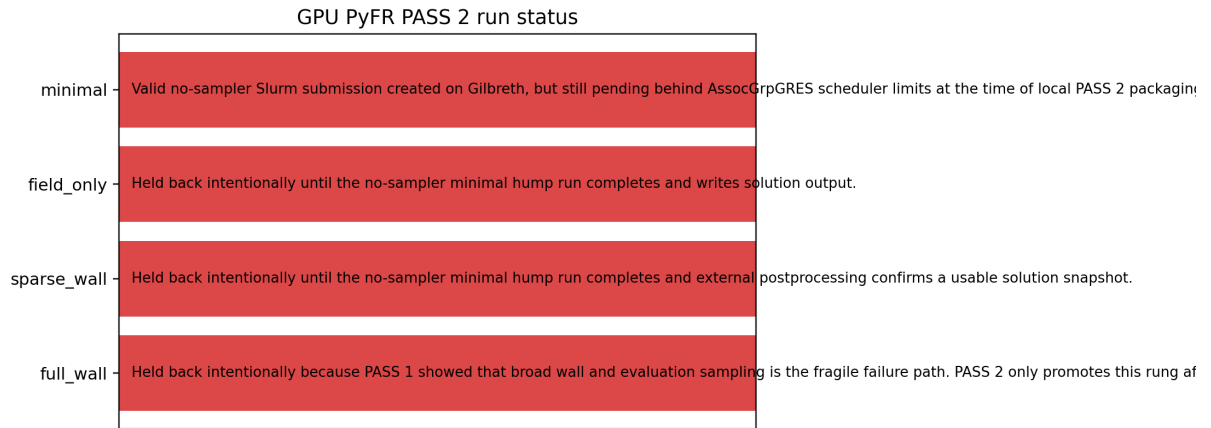


Figure 1: PASS 2 run-status summary from the staged no-sampler-to-full-wall ladder.

10 Simulation Images and Graphs

When a VTU export is available, PASS 2 includes external field plots from that snapshot.

d no-sampler Slurm submission created on Gilbreth, but still pending behind AssocGrpGRES scheduler limits at the time of local PASS 2 packagi

Figure 2: Minimal no-sampler field visualization, if VTU export succeeds.

No pseudo_stats.csv found for minimal.

Figure 3: Pseudo-stat history for the minimal PASS 2 hump run, when the corresponding CSV is available.

11 Comparison Against OpenFOAM Baseline

The OpenFOAM baseline remains stronger scientifically until PyFR produces a valid hump solution with extractable wall metrics. PASS 2 therefore compares itself honestly against the existing OpenFOAM baseline by asking:

- did the PyFR hump run complete,
- did it export a usable field snapshot,
- did it do so without the PASS 1 sampler failure.

Only if those conditions are met does it make sense to quote C_p or benchmark-style comparisons.

12 Limitations

- PASS 2 still uses `ac-navier-stokes`, not OpenFOAM SST.

- Any Cp or Cf comparison is conditional on actual solution export and defensible extraction.
- A completed no-sampler run is a robustness milestone, not an accuracy claim.

13 Reproducibility Commands

```
cd /Users/ashmitprabhu/github/nasa-hump2
ssh gilbreth
cd /scratch/gilbreth/prabhu56/nasa-hump2-gpu-pyfr-pass1/repo_git
bash scripts/setup_gilbreth_pyfr_env.sh
python3 scripts/pyfr/check_pyfr_hump_mesh.py --variant minimal
bash scripts/submit_pyfr_pass2_minimal_gilbreth.sh minimal

# Back locally
cd /Users/ashmitprabhu/github/nasa-hump2
bash scripts/fetch_gpu_pyfr_pass2_results.sh
python3 scripts/postprocess_gpu_pyfr_pass2.py
bash scripts/build_report.sh
```

14 Conclusion and Next Step

PASS 2 is successful if it breaks the PASS 1 dependency on PyFR sampler plugins and produces the first completed hump solution snapshot. If it still fails, the next step remains mesh/import robustness, not physics tuning.

References